Wizard-to-Runtime Complete Procedure

Delphi App Translation Studio

Current pre-release test workflow

Last changed: August 12, 2026

Windows - Delphi VCL and FireMonkey - Win32 and Win64

Printable, start-to-finish operator procedure

Every required transition is included: clean test copy, Wizard, localization review, second-pass update, Studio verification, RAD Studio component placement, builds, deployment, and runtime testing.

Table of Contents

1. Read This Before Starting

This procedure tests the current pre-release workflow exactly as it exists. The Setup Wizard performs the first project scan, automatic provider translation, validation, JSON export, component-kit generation, Search Path configuration, and language-pack deployment configuration. It does not place components on a Delphi form; that remains a normal RAD Studio Form Designer operation.

The current workflow requires two Wizard passes when layout or project-glossary decisions are made after the first translation. The first pass creates the translated catalog and its review proposals. The developer reviews those proposals. The second pass, Update Existing Translation, applies the saved glossary, embeds accepted layout rules in the runtime JSON, and refreshes the generated component kit. This guide never hides that second pass.

After the Wizard passes, the main Studio is used to inspect the exact development catalog, make any necessary text correction, perform final validation, export the final runtime pack, and refresh the component kit. RAD Studio is then used to install the design package, place the manager and selector, build the target, and test runtime behavior.

Do not use an original project. Create a new disposable test folder from the pristine copy. Keep the pristine folder and the original application untouched. The Wizard creates its own ZIP and DPROJ transaction backups, but those are additional safeguards, not substitutes for the pristine copy.

Current automatic-layout boundary. The runtime can apply accepted, checksum-backed Width, Height, WordWrap, and AutoSize rules from a language pack. It does not automatically move neighboring controls, redesign a form, change fonts, or guarantee that a wider control will not overlap another control. Any proposal left Pending, Rejected, or Manual is not applied automatically.

Current glossary boundary. An empty Project Glossary is normal before terms are approved. Built-in computer-interface terminology still operates. Catalog-derived suggestions appear after translations exist; only approved project terms become authoritative for that project and language.

1.1 What this test should prove

[ ] The Wizard completes without an access violation or a stopped-processing message.

[ ] The Wizard translates unresolved entries automatically with the selected API provider.

[ ] The first pass produces a development catalog, runtime pack, review package, and component kit.

[ ] The second pass preserves completed work and embeds accepted glossary/layout decisions.

[ ] The main Studio opens the same catalog, validates it, and exports the final JSON pack.

[ ] The target project contains only intentional designer changes plus the marked DPROJ configuration block.

[ ] Win32 and Win64 builds receive the correct Localization\Languages folder.

[ ] The target starts in the expected language, switches immediately, restores layout on switching back, and remembers the user's choice after restart.

[ ] Dynamic application text does not produce runaway text such as repeated o characters.

2. Exact Product Files and Generated Locations

Purpose	Path or pattern
Recommended Studio build	C:\New Delphi Projects\Delphi App Translation\bin\Win32\Debug\DelphiAppTranslationStudio.exe
Win64 Studio build	C:\New Delphi Projects\Delphi App Translation\bin\Win64\Debug\DelphiAppTranslationStudio.exe
FMX design package	C:\New Delphi Projects\Delphi App Translation\bin\packages\Win32\Release\DATLanguageManagerFMXDesign.bpl
VCL design package	C:\New Delphi Projects\Delphi App Translation\bin\packages\Win32\Release\DATLanguageManagerVCLDesign.bpl
Development catalog	<Test Project>\Localization\Development\<ApplicationId>.<language>.translation-project.json
Project glossary	<Test Project>\Localization\Glossaries\<ApplicationId>.<language>.glossary.json
Runtime language pack	<Test Project>\Localization\Languages\<language>.json
Component kit	C:\New Delphi Projects\Delphi App Translation\export\component-integration\<ApplicationId>
Localization review	C:\New Delphi Projects\Delphi App Translation\export\localization-review\<ApplicationId>\<language>
Saved language preference	%LOCALAPPDATA%\<ApplicationId>\language.ini
Wizard safety backup	%USERPROFILE%\Documents\Delphi App Translation Backups\<ApplicationId>\<timestamp>.zip

Application ID means project name, not folder name. For C:\...\Carillon.dproj the detected ApplicationId is Carillon. Always use the exact Application ID displayed by the Wizard; do not type the full path and do not include .dproj or .exe.

3. Prepare a Completely Clean Test

1. Close the target application executable if it is running.

2. Close the target project in RAD Studio. Saving is allowed before closing, but do not leave unsaved form changes.
3. Locate the pristine project folder. Confirm that it is the known-good unlocalized source.
4. Create a new sibling test folder by copying the entire pristine folder. Use a clear name such as <Application Name> - Translation Test.
5. Do not copy an earlier test folder. Earlier Localization folders, DPROJ Search Paths, components, and language preference files would invalidate the test.
6. Open the new test folder in File Explorer. Confirm the expected .dproj or .dpr file is present.
7. If the pristine copy contains old bin or dcu output folders, remove those only from the new disposable test copy or use Delphi Clean before the baseline build. Never clean the pristine source as part of this test.
8. Open the new test project's .dproj in RAD Studio, perform a baseline Win32 Debug build, and run it once in its original language.
9. Open every important form or page and confirm that the pristine application itself does not already contain mixed-language or runaway dynamic text.
10. Close the baseline executable. Choose File > Close All in RAD Studio so the target project is closed before Wizard final processing.

> **No Git requirement.** Git is useful evidence but is not required. A pristine copy plus a fresh disposable test copy is an accepted safety baseline. If Git is available, record git status --short now; it should be empty.

3.1 Clear a previous saved-language preference

A new project folder does not clear the per-user language preference because that file lives under Local AppData. If the same ApplicationId was tested previously, the new executable may immediately restore Spanish or another language even though the project folder is new.

1. Press Windows+R.
2. Enter %LOCALAPPDATA% and press Enter.
3. Open the folder whose name exactly matches the Wizard's Application ID, if it exists.
4. If language.ini exists, rename it to language.ini.pretest. Do not rename unrelated application settings.
5. If the Application ID folder does not exist, no preference cleanup is required.

4. Start the Setup Wizard

1. Run C:\New Delphi Projects\Delphi App Translation\bin\Win32\Debug\DelphiAppTranslationStudio.exe.
2. Wait for the Delphi App Translation Studio main form to appear.
3. Click Start Setup Wizard on the Project page. Do not click Open Project for this initial path.

4. Confirm the dimmed Studio background is blocked by the modal Translation Setup Wizard.
5. Do not run a second Studio instance during this test.

5. Wizard Pass One - Create the Translation

5.1 Step 1 - Welcome

1. Read the safety statement. It explains that final processing creates a backup and writes localization/configuration files but does not rewrite Pascal or form source.
2. Click Next.

5.2 Step 2 - Delphi Project

1. Click Browse.
2. Navigate to the new disposable test folder, not the pristine folder and not the original project folder.
3. Select the target .dproj. Use the .dpr only when no .dproj exists.
4. Confirm the displayed project path is the disposable test folder.
5. Confirm the detected framework is FireMonkey or VCL as expected.
6. Confirm Win32 and/or Win64 platforms and the form-resource count are plausible.
7. Write down the detected Application ID. You will use this exact value in the Object Inspector later.
8. Leave the workflow on Automatic. For a new folder it should report Create a new translation.
9. Click Next.

5.3 Step 3 - Languages

1. Set Source language to the language in which the Delphi application was authored, normally English (United States) [en-US].
2. Select exactly one Target language for this pass, for example Spanish (Spain) [es-ES].
3. Confirm Native language name is correct and uses proper characters, for example Español.
4. Confirm Direction is Left-to-right unless the language requires right-to-left text.
5. Review the date, time, decimal, thousands, and currency values. These locale settings are stored in the JSON pack; correct only values you deliberately want the target application to use.
6. Confirm Automatic still reports Create a new translation, then click Next.

> **One target language per Wizard run.** To deliver 15 languages, repeat the create/update workflow for each target language. The runtime selector later lists the source-language pack and every valid target-language JSON pack deployed beside that executable.

5.4 Step 4 - Translation Service

1. Choose Google Cloud Translation or DeepL.

2. For DeepL, choose API Free or API Pro to match the account. Google does not use the DeepL plan field.
3. If the provider key is already stored in Windows Credential Manager, leave the API key field blank and verify that the status reports a stored key.
4. Otherwise paste the API key into the masked field.
5. Leave Remember securely on this computer checked to store the key in Windows Credential Manager. Uncheck it only for a session-only key.
6. Click Save / Replace Key when a new key was entered.
7. Click Test Connection.
8. Do not continue until the status explicitly reports that the connection test passed.
9. Click Next.

> **The target application remains offline.** Only the developer's Studio uses the provider and API key. The deployed target application reads local JSON packs and does not need the Internet or the API key.

5.5 Step 5 - Scan Project

1. Click Scan Project.
2. Wait for Scan complete. Do not click Next while the scan is running.
3. Record the total translatable entries and the new/changed/unchanged/obsolete counts.
4. Scroll through representative items. Confirm form text and resourcestring entries belong to the selected project.
5. Look specifically for suspicious repeated characters, data values, paths, filenames, URLs, IDs, or logging output. These should be protected or reviewed rather than translated as normal interface text.
6. If you edit or save any target PAS, FMX, DFM, DPR, or DPROJ file after this scan, return to this step and scan again before final processing.
7. The Localization Review button is available, but the richest terminology suggestions and layout proposals do not exist until translations have been created. For the ordinary first test, continue with Next.

5.6 Step 6 - Delphi Component

1. Read the project-specific instructions from top to bottom.
2. Confirm the instructions show the correct target project path and exact Application ID.
3. Click Show Design BPL. File Explorer should select DATLanguageManagerFMXDesign.bpl for an FMX target or DATLanguageManagerVCLDesign.bpl for a VCL target under bin\packages\Win32\Release.
4. Do not use Delphi's Install Component wizard. Do not select a .dpk. The later approved procedure is Component > Install Packages > Add and selection of the compiled design BPL.
5. Return to the Wizard.

6. Check Required: I understand the remaining manual RAD Studio phase.
7. Confirm the orange reminder changes to a confirmation message.
8. Click Next.

5.7 Step 7 - Review and Authorize

1. Read the project, Application ID, framework, target language, workflow, provider, scan count, unresolved count, integration method, deployment statement, and backup statement.
2. Confirm the required backup box is checked. It is intentionally mandatory.
3. Confirm RAD Studio has no target project open. If it does, close the project now and return to the Wizard.
4. Check Required: the target project is closed in RAD Studio.
5. Check I reviewed these choices and authorize final processing.
6. Click Begin Final Processing only after both confirmations are checked.
7. After processing begins, do not try to close the Wizard or Studio. Back, Cancel, and the step rail remain disabled until processing succeeds or stops safely.

5.8 Step 8 - Processing and Completion

1. Watch the progress log until it stops changing.
2. Confirm a timestamped ZIP backup was created.
3. Confirm unresolved entries were translated by the selected provider or resolved by terminology/translation memory.
4. Confirm Development catalog saved appears.
5. Confirm localization review, layout proposal, and multilingual layout envelope paths were generated.
6. Confirm validation passed. Warnings may remain; blocking errors must not remain.
7. Confirm Runtime JSON pack exported appears with a path under the disposable test project's Localization\Languages folder.
8. Confirm Component integration kit generated appears.
9. Confirm Project Search Path and automatic post-build deployment configured appears.
10. Confirm the footer says Setup Wizard completed successfully. If it says stopped, do not continue to RAD Studio; record the complete STOPPED line and stop this test.
11. Click the blue component-kit path or Open Kit Folder. Confirm ComponentSource, Localization, component-integration.json, Deploy-LanguagePacks.ps1, README.txt, and Wizard-Completion-Report.txt exist.
12. Leave the Wizard open for the next review step. Do not click Finish yet.

6. Review the First-Pass Translation

1. On the Wizard completion page, click Review Localization.
2. On Audit & Confidence, read the summary and inspect all High risk findings and representative Warning findings.
3. Click Generate Review Package, then Open Visual Review. Review the estimated translated forms in the browser. This preview is advisory; it does not alter a Delphi form.
4. Return to the Localization Review Center.
5. Open Terminology Suggestions. Select suggestions and read Source term, Suggested target, Context, Semantic concept, Provenance, Confidence, and Why suggested.
6. Use Approve Selected only for a term you understand and want enforced for this project and language.
7. Use Approve High-confidence All only after inspecting the proposed group. Provider output is not promoted automatically merely because it exists.
8. Open Project Glossary. Confirm approved terms now appear. If a required term is absent, click New, enter Source term and Preferred translation, optionally enter Semantic concept and Developer note, leave Approved terminology checked, click Add / Update Term, and click Save Project Glossary.
9. Open Layout Proposals. Select a proposal and read its form, control, property, current value, proposed value, reason, decision, and What happens explanation.
10. For the ordinary test, click Accept All Safe Proposals only after understanding that Width, Height, WordWrap, and AutoSize rules will be applied at runtime for this language.
11. For any proposal that would collide with another control or exceed its parent, choose Manual or Rejected and click Save.
12. Click Close to return to the Wizard.
13. Click Finish to close Wizard pass one and return to the Studio main form.

> **Why pass two is required.** The first runtime pack was exported before these review decisions were made. The next Wizard update applies the saved project glossary and embeds accepted safe layout rules into a newly exported runtime pack.

7. Wizard Pass Two - Apply Saved Review Decisions

1. Click Start Setup Wizard again.
2. Click Next on Welcome.
3. Browse to the same disposable test project's .dproj.
4. Confirm the same Application ID and framework.
5. On Languages, select the same source and target language used in pass one.
6. Leave Workflow on Automatic. Confirm Existing catalog detected and Update the existing translation is reported.

7. Confirm the translation provider and stored key, then click Test Connection again.
8. Run Scan Project again. Confirm completed translations are preserved and only actual new or changed source entries are unresolved.
9. Read the component step, click Show Design BPL if desired, check the required understanding box, and click Next.
10. Close the target project in RAD Studio if it was opened accidentally.
11. Check the target-project-closed and authorization boxes, then click Begin Final Processing.
12. Confirm the progress reports glossary translations applied when applicable, validation passed, runtime pack exported, component kit generated, deployment configured, and successful completion.
13. Click Review Localization only to confirm the saved glossary and layout decisions persisted. Do not create new decisions during this confirmation pass; a new decision would require another export/update.
14. Close Localization Review and click Finish.

8. Return to the Main Translation Studio

The Wizard has already translated and exported the project. The main Studio is now used for final inspection, corrections, validation, and a final export. Do not start a second translation from scratch.

8.1 Open the matching project and catalog

1. On the Studio left rail, click 1 Project.
2. Click Open Project and select the same disposable test .dproj.
3. Click 3 Translate.
4. Click Open.
5. Open the development catalog from <Test Project>\Localization\Development. Its name is <ApplicationId>.<language>.translation-project.json.
6. Confirm the target language, native language name, locale formats, entry count, translated count, status, and origin fields are populated.
7. Click 2 Scan, then click Scan Project. This proves that the catalog still matches the saved Delphi sources.
8. Return to 3 Translate. Confirm reviewed/approved translations and existing provider results were preserved.

8.2 Inspect and correct translations

1. Select entries with short or ambiguous interface text, unknown context, provider-basic confidence, or suspicious ownership.

2. Read Source text, the context hint, runtime role, origin, and Translated text.
3. Correct an inaccurate translation in Translated text and click Apply Translation. This changes the development JSON catalog, not Delphi source.
4. Use Mark Reviewed only after a human has actually reviewed that entry. Use Approve only after it is already Reviewed.
5. Review All and Approve All are catalog-wide decisions. Do not use them merely to remove warnings; use them only when one qualified review decision truly applies to the entire displayed group.
6. Click Save after completing edits. Confirm Development catalog saved.
7. Click Translate Automatically only if the readiness line or confirmation dialog reports unresolved entries. The command sends only eligible unresolved entries; reviewed and approved entries remain unchanged.

8.3 Validate and export the final runtime pack

1. Click 4 Validation.
2. Click Run Validation.
3. If errors are reported, double-click each error, correct the referenced translation or setting, save, and run validation again. Do not export with errors.
4. Warnings do not block export. Review warnings about identical source/translation, placeholders, accelerator counts, runtime Translate calls, and suspicious text; document why any accepted warning is safe.
5. When the summary reports 0 errors, click 5 Export.
6. Click Export Runtime Pack.
7. Confirm the runtime entry count and click the blue output path.
8. In File Explorer, confirm the final <language>.json exists under the disposable test project's Localization\Languages folder.
9. Open the JSON in a text-safe viewer. Confirm schemaVersion is 3 and confirm a layout array is present when safe layout proposals were accepted.

8.4 Refresh the component kit after the final export

1. Click 6 Integration.
2. Confirm Integration method is Component Integration (Recommended).
3. Click Build Integration Plan.
4. Confirm the target framework, translated pack count, generated English pack statement, manager class, and selector class are correct.
5. Click Generate Component Kit.
6. Click Open Kit Folder and confirm the kit was refreshed. This step is important because the kit must contain the same final runtime JSON pack that you just exported.

7. Click Show Design BPL and leave File Explorer open with the exact Win32 Release design BPL selected.

9. Install the Design Package in RAD Studio

> **Manual installation is intentional.** The product does not automatically register a design-time BPL. Manual installation through RAD Studio's approved package dialog avoids changing the IDE behind the developer's back.

1. Start RAD Studio without opening the target project or target form.
2. Choose Component > Install Packages.
3. If DAT Language Manager FireMonkey design-time package or the VCL equivalent is already listed and checked, do not add a duplicate. Click OK and continue to the next section.
4. Otherwise click Add.
5. Browse to the exact BPL selected by Show Design BPL: DATLanguageManagerFMXDesign.bpl for FMX or DATLanguageManagerVCLDesign.bpl for VCL.
6. Select the .bpl and click Open. Do not choose a .dpk and do not use Component > Install Component.
7. Confirm the DAT design-time package appears in the Design packages list and is checked.
8. Click OK.
9. Create or open a harmless blank form if necessary and confirm the Tool Palette contains DAT Localization with both the language manager and language combo box.

10. Place and Configure the Components

1. Open the disposable target project's .dproj in RAD Studio.
2. Open the primary form in the Form Designer. If Delphi opened it automatically, simply confirm that the primary form is the active designer.
3. From DAT Localization, place one TDATFMXLanguageManager for FMX or TDATVCLLanguageManager for VCL on the primary form.
4. Select the manager in the Object Inspector.
5. Set ApplicationId to the exact value displayed by the Wizard.
6. Leave LanguagesFolder as Localization\Languages.
7. Set SourceLanguage to the source language code used by the Wizard, normally en-US.
8. For a clean first-launch test, set AutoLoadPreferred to False. This forces the source language initially. After first-launch behavior passes, set it back to True to test saved-user preference behavior.
9. Leave AutoTranslateOwner, AutoTranslateNewForms, ReapplyOpenForms, and PreserveControlState enabled unless this test intentionally exercises another setting.
10. Place one TDATFMXLanguageComboBox or TDATVCLLanguageComboBox on the visible primary form. The selector is required unless the application supplies a connected Language menu.

11. Select the combo box. In the Object Inspector, set LanguageManager to the manager component you just placed. Do not leave it blank.
12. Position and size the combo box where it is visible and does not cover existing controls. Add a normal designer-authored label such as Language: if the application needs one.
13. Choose File > Save All. This save is essential; it writes the two designer components and their properties to the form resource and updates the project metadata as required by Delphi.

One manager, not one per form. Place the manager on the primary form only. It observes and translates other open or newly created forms. Ordinary secondary forms do not need another manager component.

11. Verify Search Path and Build Configuration

1. Choose Project > Options.
2. Select Building > Delphi Compiler > Search path.
3. Inspect Value from All configurations. It should contain the generated kit's ComponentSource folder under C:\New Delphi Projects\Delphi App Translation\export\component-integration\<ApplicationId>\ComponentSource.
4. Use the Target selector to inspect Debug/Release and Windows 32-bit/64-bit as applicable. The Wizard-created DPROJ block is intended to cover all configurations and platforms.
5. If the path is absent, stop. Do not manually invent a different path during this acceptance test; record the missing configuration because the Wizard was supposed to add it.
6. Click Cancel in Project Options when no manual change is necessary.

12. Build Win32 and Win64

1. Select Win32 and Debug in Project Manager, then choose Project > Build <ApplicationId>.
2. Confirm the build completes with zero fatal errors.
3. Locate the Win32 Debug executable using Project > Options > Building > Delphi Compiler > Output directory if the project uses a nonstandard path.
4. Beside the executable, confirm Localization\Languages exists and contains en-US.json plus the target-language JSON file.
5. Repeat for Win64 Debug.
6. If release testing is required, repeat for Win32 Release and Win64 Release.
7. After every build, verify the executable's own folder contains Localization\Languages. The project-root Localization folder alone is not sufficient for runtime discovery.

The .rsm file is normal. A Delphi build may create both <ApplicationId>.exe and <ApplicationId>.rsm. The .exe is the application. The .rsm contains debug symbol information and is not a language pack.

12.1 Manual deployment fallback

Use this only if automatic post-build deployment did not create the language folder. Substitute the actual kit and executable folder paths shown by the Wizard. The command explicitly uses ExecutionPolicy Bypass.

```
& "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -NoProfile -ExecutionPolicy
Bypass -File "C:\New Delphi Projects\Delphi App Translation\export\component-
integration\<ApplicationId>\Deploy-LanguagePacks.ps1" -ApplicationDirectory "<Folder
containing ApplicationId.exe>"
```

A successful command reports Language packs deployed to <application folder>\Localization\Languages. The PowerShell process should exit when the command finishes.

12.2 Portable or USB executable

1. Copy or build the final executable into the desired portable folder or USB drive.
2. Run the fallback deployment command above with -ApplicationDirectory set to the full folder containing the executable, including the drive letter such as F:\Carillon Portable.
3. Confirm <Portable Folder>\Localization\Languages contains en-US.json and every target-language JSON file.
4. Keep the manager's LanguagesFolder property relative as Localization\Languages. Do not put F: or another drive letter in the component property.

13. Runtime Test - First Launch and Switching

1. Confirm the target application is closed.
2. Confirm AutoLoadPreferred is False for this first-launch test, or confirm the prior language.ini was renamed as described in Section 3.1.
3. Run the Win32 Debug executable directly from its output folder.
4. Confirm the application initially displays its source language and the selector shows the source language.
5. Open the selector. Confirm it contains the source language plus one item for every deployed valid target pack. Fifteen target packs should produce sixteen choices when the source pack is included.
6. Select the target language.
7. Confirm visible designer text changes immediately without restarting the application.
8. Open each secondary form and visit each page or tab. Confirm forms created after the selection also appear in the active language.
9. Return to the primary form and select the source language.
10. Confirm source text and original layout values are restored.

11. Repeat source-to-target-to-source switching at least ten times. Confirm controls do not grow cumulatively and the application does not lose dates, selections, focus, connection state, or editable values.
12. Close the executable.
13. Set AutoLoadPreferred back to True in the Object Inspector, Save All, and rebuild the configuration.
14. Run the rebuilt executable, select the target language, close it, and run it again.
15. Confirm it restores the user's last selected language. This is expected saved-preference behavior, not corruption of the source application.
16. Repeat representative switching and restart checks with Win64 Debug and required Release builds.

14. Runtime Layout and Content Acceptance

14.1 What accepted layout rules should do

[ ] Accepted Width or Height rules apply only when their target language is active.

[ ] Accepted WordWrap or AutoSize rules apply only to the identified control and language.

[ ] Switching away restores the original property value before another language is applied.

[ ] Repeated language changes do not compound Width or Height.

[ ] No target PAS, FMX, DFM, DPR, or DPROJ form layout is rewritten by a runtime layout rule.

14.2 What still requires human inspection

[ ] A wider translated control does not overlap its neighbor or leave its parent container.

[ ] Wrapped labels have enough height and remain aligned with related controls.

[ ] Buttons, tabs, menu items, column headers, charts, grids, and status areas remain readable.

[ ] Dynamic text produced by application code uses Translate or FormatTemplate where required.

[ ] The translation is correct in application context, not merely a valid dictionary translation.

[ ] Form titles, secondary forms, help text, and runtime status messages have each been considered separately.

15. Diagnose Missing, Random, or Runaway Text

1. Record the form, page, control, active language, exact displayed text, and a screenshot.
2. Switch to the source language. Determine whether the same bad text exists in the pristine baseline application.

3. If the problem exists in the pristine baseline, classify it as a target-application defect rather than a translation defect.
4. If the source language is correct but the target language is wrong, search the development catalog for the control key or source text.
5. If the entry exists, inspect its source ownership, runtime role, context, translation, and status. Correct the translation or runtime classification as appropriate.
6. If the entry does not exist, save all target files, close the target project, reopen it in the Studio, scan again, and record whether the scan count increases.
7. For runtime-generated text, verify that the target application uses the manager's Translate or FormatTemplate API with the catalog key. Static form scanning cannot translate arbitrary text assembled later by application code.
8. For repeated o characters or other actively growing output, stop the executable. Record whether the source-language run also reproduces it. A JSON translation pack supplies a fixed string; unbounded growth usually indicates an application timer, refresh, append, or data-writing loop and must be isolated separately.
9. After any catalog correction, Save, Run Validation, Export Runtime Pack, regenerate the component kit, rebuild/deploy, and retest.

16. Procedure After Later UI Changes

> **Batch changes when practical.** Add or revise labels, buttons, menus, headings, hints, and resourcestrings in one saved batch when possible. This reduces repeated scans and provider calls, but the same procedure works for one change.

1. In RAD Studio, make the designer or source changes in the target application.
2. Choose File > Save All.
3. Close the running target executable.
4. Close the target project in RAD Studio before any Wizard final-processing pass.
5. Open the project in the Translation Studio.
6. Open the existing development catalog before scanning.
7. Run Scan Project.
8. Confirm new entries are New, revised source text is Source Changed, unchanged completed translations remain preserved, and removed items become Obsolete.
9. Run Translate Automatically. Confirm only eligible unresolved/new/changed entries are offered to the provider.
10. Review the new translation and any new terminology/layout proposal.
11. Save the catalog, run validation, and resolve all errors.
12. Export Runtime Pack.
13. Regenerate the Component Kit so its Localization folder contains the final pack.

14. Build the target; confirm post-build deployment refreshes Localization\Languages beside the executable.
15. Run the target and retest the changed form in the source and target language.

17. Adding More Languages

1. Keep the same disposable or approved working project and its existing manager/selector components.
2. Start the Setup Wizard.
3. Select the same project and source language but choose a different target language.
4. Use Automatic. It should create a new development catalog for that language while preserving catalogs for other languages.
5. Complete Wizard pass one, review terminology and layout for that language, and complete Wizard pass two when decisions were saved.
6. Perform final Studio validation/export and regenerate the component kit.
7. Build or deploy the packs to every executable folder.
8. Run the application. The existing connected selector should discover the new JSON pack automatically; do not add another manager or combo box.
9. Test translation, layout, source restoration, and preference behavior separately for the new language.

18. Final Completion Checklist

[ ] Disposable test folder was copied from pristine and the pristine/original folders remained untouched.

[ ] Wizard pass one completed successfully and translated unresolved entries.

[ ] Localization audit, terminology suggestions, glossary, and layout proposals were reviewed.

[ ] Wizard pass two applied saved review decisions.

[ ] Main Studio opened the matching catalog, rescanned, validated with zero errors, and exported the final runtime pack.

[ ] Component kit was regenerated after the final export.

[ ] Correct Win32 Release design BPL was installed through Component > Install Packages > Add.

[ ] One manager and one connected visible selector were placed and saved on the primary form.

[ ] ApplicationId, LanguagesFolder, SourceLanguage, and LanguageManager properties were verified in Object Inspector.

[ ] Search Path contains the generated ComponentSource folder for all required configurations/platforms.

[ ] Win32 and Win64 builds completed.

[ ] Each executable folder contains Localization\Languages with en-US.json and all target packs.

[ ] First launch used the source language under the controlled test condition.

[ ] Immediate switching, secondary forms, source restoration, repeated switching, and restart preference passed.

[ ] Accepted safe layout rules worked without cumulative growth or unacceptable overlap.

[ ] No missing/random translation or runaway dynamic-text defect remains unexplained.

19. What to Send Back After Testing

- Disposable test-folder path and selected .dproj path.
- Application ID, framework, source language, target language, and provider.
- Pass-one and pass-two scan totals and unresolved counts.
- Wizard completion log or the exact STOPPED message.
- Validation error/warning/information counts.
- Number of glossary terms approved and layout proposals accepted/manual/rejected.
- Whether the final runtime JSON has schemaVersion 3 and a layout array.
- Win32/Win64 build results and exact executable/output folders.
- Contents of each executable's Localization\Languages folder.
- First-launch language, selector choices, restart language, and ten-cycle switching result.
- Screenshots and exact control/form names for every untranslated, mistranslated, truncated, overlapping, or runaway item.

20. Important Stop Conditions

- Stop if the selected project path is the pristine or original application.
- Stop if Wizard final processing reports STOPPED or validation reports blocking errors.
- Stop if the design package fails to load; do not remove unrelated Delphi packages.
- Stop if the Wizard Search Path is absent rather than masking the defect with an unrelated global path.
- Stop if runtime packs beside the executable do not match the Application ID or selected language.
- Stop a target executable that generates unbounded repeated text; preserve evidence before restarting.

- Do not distribute or publish this pre-release test output as a production localization solution.